Лабораторна робота №2

Тема: Розробка ноди керування рухом робота через /cmd_vel

Мета
Сформувати практичні навички розробки ноди керування рухом у ROS2, навчитися працювати з повідомленням geometry_msgs/Twist, реалізувати підписку на /cmd_vel, обмеження швидкості та перевірити роботу ноди в симуляції.

Необхідне обладнання

- ПК або ноутбук
- Віртуальна машина з Ubuntu
- Встановлений ROS2 (Humble або узгоджена версія)
- IDE (VS Code / PyCharm / інша)
- Workspace, створений у Лабораторній роботі №1
- Доступ до OneDrive або Git для здачі результатів

Завдання

1. Підготовка середовища

1.1. Активувати ROS2-середовище.
1.2. Перевірити працездатність команд:
    - ros2 node list
    - ros2 topic list
1.3. Запустити turtlesim або інший доступний симулятор для тестування.

2. Створення пакету Motion Control

2.1. Створити новий пакет типу ament_python для реалізації ноди керування рухом.
2.2. Додати залежність від geometry_msgs.
2.3. Зібрати workspace та перевірити коректність встановлення пакету.

3. Реалізація Motion Node

3.1. Створити ноду, яка підписується на Topic:
/cmd_vel
3.2. Використати тип повідомлення:
geometry_msgs/msg/Twist
3.3. Реалізувати обробку полів:
    - linear.x
    - angular.z
3.4. Виводити отримані значення через логер ROS2 (INFO-рівень).

4. Реалізація обмеження швидкості

4.1. Додати перевірку допустимих меж швидкості.
4.2. Якщо значення перевищує встановлений поріг — обмежити його.
4.3. У разі перевищення виводити WARN-повідомлення.

5. Тестування ноди

5.1. Запустити Motion Node.

5.2. Подати команди у /cmd_vel через teleop або через ros2 topic pub.

5.3. Перевірити, що нода отримує повідомлення та коректно їх обробляє.

5.4. Переконатися, що обмеження швидкості працює.

Вимоги до звіту (згідно Додатку 1)

Звіт повинен містити:

1. Опис реалізації Motion Node.
2. Фрагменти коду підписки на /cmd_vel.
3. Скриншоти:
    - запуску ноди
    - виводу логів
    - перевірки через ros2 topic echo
4. Демонстрацію обмеження швидкості (скриншот WARN-повідомлення).
5. Висновки щодо різниці між “ручним керуванням” та програмним керуванням.

Додаток1

Структура здачі

```
ROS2_Прізвище/
└── Lab2/
    ├── screenshots/
    ├── code/
    └── report.pdf або report.md
```

Теоретичні відомості

1. Архітектура ROS2: Publisher / Subscriber.
2. Topic /cmd_vel як стандарт керування мобільними роботами.
3. Тип повідомлення geometry_msgs/Twist.
4. Лінійна та кутова швидкість.
5. Принцип диференціального керування.
6. Основи безпечного обмеження швидкості.

Очікуваний результат

Після виконання лабораторної роботи студент повинен:

- розуміти принцип керування рухом через /cmd_vel;
- вміти реалізовувати ноду-підписник;
- застосовувати перевірку та обмеження швидкості;
- розуміти важливість безпечного керування у робототехніці;
- бути готовим до перенесення ноди на реальний робот у аудиторії.

Додаток2

📚 **Література до Лабораторної роботи №2**
(Розробка ноди керування рухом через /cmd_vel)

Основна література

1. Open Robotics. *geometry_msgs/Twist Message Specification*.
https://docs.ros.org/en/humble/p/geometry_msgs/msg/Twist.html
2. Open Robotics. *Writing a Simple Publisher and Subscriber (Python)*.
https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html
3. Siegwart R., Nourbakhsh I., Scaramuzza D.
Introduction to Autonomous Mobile Robots. MIT Press, 2011.

Додаткова література

4. Siciliano B., Khatib O. (eds.).
Springer Handbook of Robotics. Springer, 2016.
5. Диференціальна кінематика мобільних роботів (огляд):
https://www.roboticsbook.org
6. ROS2 CLI tools documentation:
https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html